34. Reverse First K Items of a Queue in Java — Quick Notes

Reverse the first `k` elements of a queue while keeping the remaining elements in the same order.

Problem Statement

Given:
Queue q
Integer k

Reverse: First k elements
Modify the **same queue**.

Example 1

Input: Queue: 10 20 30 40 50
k = 3
Output: 30 20 10 40 50

Example 2

Input: Queue: 10 20 30 40 50
k = 4
Output: 40 30 20 10 50

Key Observation

Queue provides: FIFO
But reversing requires: LIFO
Use: Stack

Approach

Step 1

Move first `k` queue elements into stack.
Queue:
10 20 30 40 50
↓
Stack:
30
20
10

Queue:
40 50

Step 2

Pop stack elements back into queue.
Queue: 40 50 30 20 10

Step 3

Move first `(n-k)` elements to rear.
40 → rear

50 → rear

Result: 30 20 10 40 50

Algorithm

1. Validate k.
2. Push first k queue elements into stack.
3. Pop stack elements back into queue.
4. Rotate first (n-k) elements.

Java Implementation

```
import java.util.*;

class Solution {
  static void reverseK(Queue<Integer> q, int k) {
      if (k > q.size() || k <= 0)
          return;

      Deque<Integer> stack = new ArrayDeque<>();
      int n = q.size();

      // Move first k items to stack
      for (int i = 0; i < k; i++) {
          stack.push(q.poll());
      }

      // Add them back to queue
      while (!stack.isEmpty()) {
          q.offer(stack.pop());
      }

      // Fix remaining order
      for (int i = 0;i < n - k; i++) {
          q.offer(q.poll());
      }
  }
}
```

Dry Run

Input:
Queue: 10 20 30 40 50
k = 3

Initial

Front

10 20 30 40 50

Rear

Push first 3 into stack

Stack:
30
20
10
Queue: 40 50

Pop stack into queue
Queue: 40 50 30 20 10

Rotate n-k = 2 times
Move 40
50 30 20 10 40

Move 50
30 20 10 40 50

Final Output
30 20 10 40 50

Why Rotate?
After Step 2: 40 50 30 20 10

The reversed elements are correct: 30 20 10

but they are at the rear.
Moving the first: n - k
elements to the rear shifts them into their correct position.

Invalid Cases
```
if(k > q.size() || k <= 0)
  return;
```

---

k > size
Cannot reverse more elements
than present.

---

k ≤ 0
Nothing to reverse.

Complexity Analysis

Step 1
k times:

stack.push(q.poll())
Complexity: Θ(k)

Step 2
k times: q.offer(stack.pop())
Complexity: Θ(k)

Step 3
n-k times: q.offer(q.poll())
Complexity: Θ(n-k)

Total Time Complexity
Θ(k) + Θ(k) + Θ(n-k) = Θ(n+k)
Since: k ≤ n
Therefore: Θ(n)

Space Complexity
Stack stores: k elements
Therefore: Θ(k)

Interview Thinking Process

Need to reverse first k.
↓
Queue cannot reverse.
↓
Stack reverses order.
↓
Use stack for first k.
↓
Need remaining elements
in same order.
↓
Rotate n-k elements.

Interview Comment (2 lines)
// Use a stack to reverse the first k queue elements.
// Rotate the remaining n-k elements to restore their order.

Applications
Used in problems involving:
- Partial queue reversal
- Buffer manipulation
- Scheduling systems
- Queue transformations

Quick Revision

Concept	Remember
Goal	Reverse first k queue elements
Extra DS Used	Stack
Step 1	Queue → Stack
Step 2	Stack → Queue
Step 3	Rotate n-k elements
Invalid Case	`k > size` or `k <= 0`
Time Complexity	Θ(n)
Auxiliary Space	Θ(k)

Template

```
for(int i=0;i<k;i++)
   stack.push(q.poll());

while(!stack.isEmpty())
   q.offer(stack.pop());

for(int i=0;i<n-k;i++)
   q.offer(q.poll());
```

This is the standard **O(n)** solution for reversing the **first K elements of a queue**.